NETWORK AUTOMATION LAB GUIDE

EVE-NG · Ansible · NetBox · Intent-Based Networking

A Complete Walkthrough from Manual CLI to Intent-Driven Automation

Lab Engineering Track | Enterprise Automation Series

Lateef Law

Technology Enthusiast

CCIE Enterprise | CCIE Service Provider

Table of Contents

1. Lab Overview & Objectives....4
1.1 Automation Maturity Model....4
2. Lab Topology....6
2.1 EVE-NG Node Images Required....6
2.2 Network Design....6
2.3 Device Addressing....7
2.4 Interface Assignments....7
3. Router Baseline Configuration....8
3.1 R1-BRANCH....8
3.2 R2-HQ....9
3.3 R3-DC....10
4. Automation Box Setup....11
4.1 System Dependencies....11
4.2 Project Structure & Directory Layout....11
4.3 Python & Ansible Packages....12
4.4 ansible.cfg....12
4.5 Baseline hosts.yml....13
4.6 Verify Reachability and Ansible Inventory....13
4.7 group_vars/all.yml....16
5. NetBox Deployment....17
5.1 Install NetBox via Docker....17
5.2 Create API Token....17
5.3 Populate NetBox Data Model....18
6. Dynamic NetBox Inventory (Lab 8)....19
6.1 Install NetBox Ansible Plugin....19
6.2 inventories/netbox.yml....19
6.3 Verify Dynamic Inventory....20
7. The Automation Progression — Theory & Context....21
7.1 Lab 1 — Level 1: Manual CLI....21
7.2 Lab 2 — Level 2: Scripts (Ad-hoc Automation)....22
7.3 Lab 3 — Level 3: Ansible Playbooks (Task-Based Automation)....25
7.4 Lab 4 — Level 4: Ansible + Templates (Abstraction)....27
7.5 Lab 5 — Level 5: Ansible + NetBox (Source of Truth)....29
7.6 The Core Mental Model....33

8. Level 3 Labs — Ansible Playbooks (Labs 6 & 7)....35
8.1 Lab 6 — Loopback Deployment....35
8.2 Lab 7 — Configuration Backup....36
9. Level 5 Labs — NetBox-Driven Configuration (Lab 9)....38
9.1 NTP Configuration from NetBox....39
9.2 Lab 9 — Interface Descriptions from NetBox....42
10. Lab Summary....44
11. Continuing the Journey — Labs 10 Through 13....45
Lab 10 — Full Intent Model (Level 5 Advanced)....45
Lab 11 — Compliance Checks and Tag-Based Enforcement (Level 5 Advanced)....49
Lab 12 — CI/CD Pipeline with Git (Level 5 / Bridge to Level 6)....52
Lab 13 — Drift Detection and Remediation (Level 5 / Bridge to Level 6)....55
12. Troubleshooting Reference....59
12.1 Common Errors & Fixes....59
12.2 Key Architecture Lessons....59
Appendix A — Installing EVE-NG Node Images with FileZilla....60
A.1 Prerequisites....60
A.2 Connect to EVE-NG via FileZilla....60
A.3 Upload C8000V Router Image....60
A.4 Upload Catalyst 9KV Switch Image....61
A.5 Upload Ubuntu Linux Image (AutomationBox)....61
A.6 Verify Images in EVE-NG....61

1. Lab Overview & Objectives

This lab builds a complete network automation platform from the ground up. Starting with manual CLI configuration of Cisco IOS-XE routers in EVE-NG, you progressively automate every aspect of network operations culminating in intent-based deployments driven entirely by data from NetBox.

By the end of this lab you will have built and operated:

- A multi-router EVE-NG topology with real WAN, LAN, and management segments
- A full Ansible automation environment on a Linux automation box
- Playbooks for loopback deployment, configuration backup, OSPF deployment and more
- A NetBox instance deployed as a Source of Truth
- Dynamic inventory driven by the NetBox API
- Intent-based deployments driven entirely by data from NetBox providing zero hardcoding

Why This Matters

The skills built here reflect real enterprise automation workflows. Rather than running show commands by hand, you will understand how organizations at scale deploy, validate, and maintain network configurations programmatically with NetBox as the authoritative data source.

1.1 Automation Maturity Model

This lab walks through seven levels of network automation maturity. Levels 1 through 5 are covered in detail with working code examples in Section 7. These are the levels you will build hands-on in this lab. Levels 6 and 7 describe the next phases involving Cisco NSO and model-driven orchestration, which represent the direction the industry is heading. These levels are the focus of Lab Guide Part 2. The table below gives a quick reference for all seven levels.

Level	Stage	What it solves
1	Manual CLI	Baseline - commands typed by hand on each device individually
2	Scripts (Ad-hoc)	Speed - any scripting language (Python/Paramiko, Bash/SSH, Netmiko, NAPALM, expect) used to push CLI commands. Faster than manual but fragile, no idempotency, no structure
3	Ansible Playbooks	Task-based automation - purpose-built IOS modules with idempotency, structured inventory, and run reporting built in
4	Ansible + Templates	Separation of concerns - Jinja2 templates hold logic, inventory holds data
5	Ansible + NetBox	Source of truth - NetBox API replaces all static YAML, zero hardcoding

Level	Stage	What it solves
6	Service Orchestration (NSO)	Stateful service modeling - NSO maintains the relationship between service intent and device config. Supports drift detection, atomic rollback, and multi-device transactions. Ansible pushes and forgets; NSO remembers.
7	Model-Driven / Full Intent	YANG data models define desired network state abstractly. The orchestrator translates intent into vendor-specific config across any platform. The operator says what they want and the system figures out every command required.

2. Lab Topology

2.1 EVE-NG Node Images Required

Before building the topology in EVE-NG, ensure the following node images are installed on your EVE-NG server. These are the exact images used in this lab. Version numbers matter for feature and interface compatibility.

Node	EVE-NG Image Name	Template Type	Used For
R1-BRANCH, R2-HQ, R3-DC	8000V-17.13.01-CML	Cisco C8000V	WAN routers
SW-LAN1, SW-LAN2, SW-LAN3	CAT9KV-17.13.1	Cisco Catalyst 9KV	LAN switches
Host-A, Host-B, Host-C	VPCS (built-in)	VPCS	End hosts
AutomationBox	Linux-ubuntu-22.04-server	linux	Ansible + NetBox

Installing images in EVE-NG

C8000V, Catalyst 9KV, and Ubuntu Linux images are all uploaded to the EVE-NG server using FileZilla (SFTP), see Appendix A for step-by-step instructions. VPCS is built into EVE-NG and requires no installation. The AutomationBox uses the Linux template with the linux-ubuntu-22.04-server image. This must be installed on the EVE-NG server before adding the node. See Appendix A for the full walkthrough.

2.2 Network Design

The lab uses a three-router triangle topology designed to simulate a real enterprise WAN with branch, headquarters, and data center nodes. Each router has a dedicated management interface, WAN links to its peers, and a LAN segment representing downstream users.

NOTE: The management segment should tie into your local network, if you prefer to connect directly to your lab environment, and therefore will be different than the management network displayed below.

Segment	Network / Detail
Management	10.1.6.0/24 (will be specific to your local network)
WAN R1–R2	10.0.12.0/30
WAN R1–R3	10.0.13.0/30
WAN R2–R3	10.0.23.0/30
LAN R1-BRANCH	192.168.10.0/24
LAN R2-HQ	192.168.20.0/24

LAN R3-DC	192.168.30.0/24

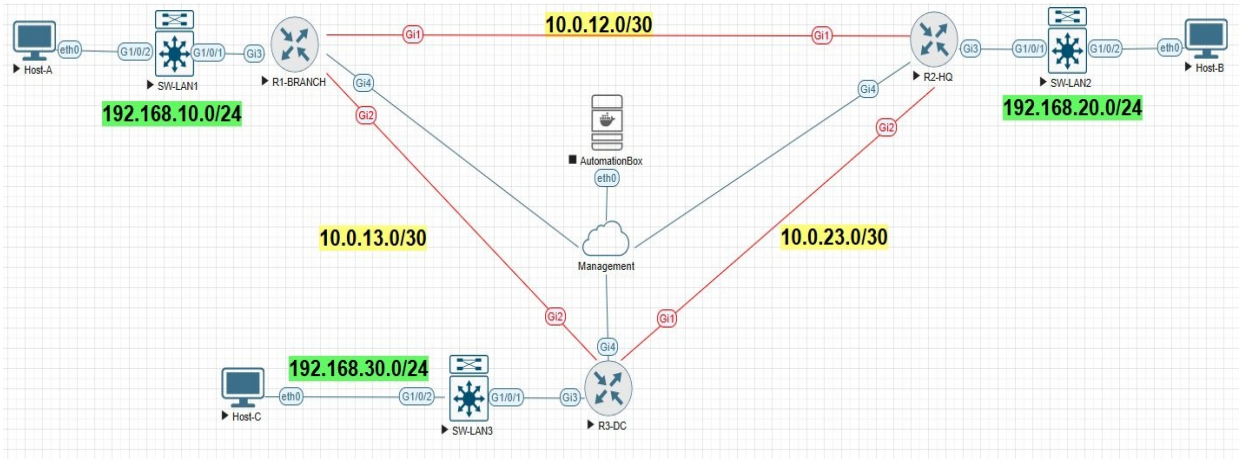


Figure 1 — EVE-NG lab topology: R1-BRANCH, R2-HQ, R3-DC, AutomationBox, Management cloud, LAN switches, and host endpoints

2.3 Device Addressing

Device	Management IP
Automation Box	10.1.6.10
R1-BRANCH	10.1.6.101
R2-HQ	10.1.6.102
R3-DC	10.1.6.103

2.4 Interface Assignments

Each router uses a consistent interface convention throughout the lab:

Interface	Purpose
GigabitEthernet1	WAN link (varies per router)
GigabitEthernet2	WAN link (varies per router)
GigabitEthernet3	LAN segment
GigabitEthernet4	Management (10.1.6.x/24) — excluded from OSPF
Loopback0	Router ID / loopback address
Loopback100	Lab 6 automation test interface

3. Router Baseline Configuration

Before automation begins, each router requires a baseline configuration applied manually via the EVE-NG console. This establishes SSH access, which Ansible will use for all subsequent automation, and pre-configures all interfaces so the network is physically reachable before any playbook runs.

The sections below show the complete baseline for each router. Interfaces are configured as follows:

NOTE: The IP's assigned to interface GigabitEthernet4 (Mgmt) will be specific to your local network.

Interface	R1-BRANCH	R2-HQ	R3-DC
GigabitEthernet1 (WAN)	10.0.12.1/30	10.0.12.2/30	10.0.23.2/30
GigabitEthernet2 (WAN)	10.0.13.1/30	10.0.23.1/30	10.0.13.2/30
GigabitEthernet3 (LAN)	192.168.10.1/24	192.168.20.1/24	192.168.30.1/24
GigabitEthernet4 (Mgmt)	10.1.6.101/24	10.1.6.102/24	10.1.6.103/24
Loopback0 (Router-ID)	1.1.1.1/32	2.2.2.2/32	3.3.3.3/32

3.1 R1-BRANCH

Apply via the EVE-NG console before connecting Ansible.

NOTE: The lab may already have the following baseline configuration. Verify first.

```
hostname R1-BRANCH
no ip domain-lookup
ip domain name lab.local
!
username admin privilege 15 secret Cisco123!
!
crypto key generate rsa modulus 2048
ip ssh version 2
!
line vty 0 4
 login local
 transport input ssh
!
interface Loopback0
 ip address 1.1.1.1 255.255.255.255
 no shutdown
```

```
!
interface GigabitEthernet1
 description WAN-to-R2-HQ
 ip address 10.0.12.1 255.255.255.252
 no shutdown
!
interface GigabitEthernet2
 description WAN-to-R3-DC
 ip address 10.0.13.1 255.255.255.252
 no shutdown
!
interface GigabitEthernet3
 description LAN-BRANCH
 ip address 192.168.10.1 255.255.255.0
 no shutdown
!
interface GigabitEthernet4
 description MANAGEMENT
 ip address 10.1.6.101 255.255.255.0
 no shutdown
!
end
write memory
```

3.2 R2-HQ

```
hostname R2-HQ
no ip domain-lookup
ip domain name lab.local
!
username admin privilege 15 secret Cisco123!
!
crypto key generate rsa modulus 2048
ip ssh version 2
!
line vty 0 4
 login local
 transport input ssh
!
interface Loopback0
 ip address 2.2.2.2 255.255.255.255
 no shutdown
!
interface GigabitEthernet1
 description WAN-to-R1-BRANCH
 ip address 10.0.12.2 255.255.255.252
 no shutdown
!
interface GigabitEthernet2
 description WAN-to-R3-DC
 ip address 10.0.23.1 255.255.255.252
 no shutdown
!
interface GigabitEthernet3
 description LAN-HQ
 ip address 192.168.20.1 255.255.255.0
 no shutdown
!
interface GigabitEthernet4
```

```
 description MANAGEMENT
 ip address 10.1.6.102 255.255.255.0
 no shutdown
!
end
write memory
```

3.3 R3-DC

```
hostname R3-DC
no ip domain-lookup
ip domain name lab.local
!
username admin privilege 15 secret Cisco123!
!
crypto key generate rsa modulus 2048
ip ssh version 2
!
line vty 0 4
 login local
 transport input ssh
!
interface Loopback0
 ip address 3.3.3.3 255.255.255.255
 no shutdown
!
interface GigabitEthernet1
 description WAN-to-R2-HQ
 ip address 10.0.23.2 255.255.255.252
 no shutdown
!
interface GigabitEthernet2
 description WAN-to-R1-BRANCH
 ip address 10.0.13.2 255.255.255.252
 no shutdown
!
interface GigabitEthernet3
 description LAN-DC
 ip address 192.168.30.1 255.255.255.0
 no shutdown
!
interface GigabitEthernet4
 description MANAGEMENT
 ip address 10.1.6.103 255.255.255.0
 no shutdown
!
end
write memory
```

Validation

After applying baseline config on each router, verify SSH connectivity from the automation box before proceeding: ssh admin@<R1-BRANCH mgmt. IP address>. If it connects, Ansible can reach the device. Repeat for R2-HQ mgmt. IP address and R3-DC mgmt. IP address. Also verify L3 reachability across WAN links: ping 10.0.12.2 from R1-BRANCH should succeed before running any playbooks.

4. Automation Box Setup

The automation box (10.1.6.10) is an Ubuntu Linux VM that runs Ansible and all supporting Python tooling. All commands in this section are run on the automation box, not the routers.

4.1 System Dependencies

Run the following commands on the automation box to install all required system packages:

```
sudo apt-get update
sudo apt install -y python3-venv python3-pip sshpass nano
```

4.2 Project Structure & Directory Layout

Create the project directory, set up the virtual environment, and scaffold the full directory structure in one sequence:

```
mkdir -p network-automation/backups \
         network-automation/templates \
         network-automation/playbooks \
         network-automation/inventories/lab/group_vars \
         network-automation/inventories/lab/host_vars && \
touch network-automation/ansible.cfg \
      network-automation/templates/ospf.j2 \
      network-automation/playbooks/deploy_ospf.yml \
      network-automation/playbooks/deploy_ospf_from_template.yml \
      network-automation/playbooks/deploy_ospf_from_netbox.yml \
      network-automation/playbooks/backup_configs.yml \
      network-automation/playbooks/deploy_loopback.yml \
      network-automation/inventories/netbox.yml \
      network-automation/inventories/lab/hosts.yml \
      network-automation/inventories/lab/group_vars/all.yml \
      network-automation/inventories/lab/host_vars/R1-BRANCH.yml \
      network-automation/inventories/lab/host_vars/R2-HQ.yml \
      network-automation/inventories/lab/host_vars/R3-DC.yml
```

Create and activate the virtual environment:

```
cd ~/network-automation
python3 -m venv automation-venv
source automation-venv/bin/activate
```

Note: All commands from this point forward assume the virtual environment is active. If you close your terminal or return to the lab after a break, reactivate it before continuing:

The resulting structure looks like this:

```
network-automation/
├── ansible.cfg
├── backups/
├── templates/
│   └── ospf.j2
├── playbooks/
│   ├── deploy_ospf.yml
│   ├── deploy_ospf_from_template.yml
│   ├── deploy_ospf_from_netbox.yml
│   ├── backup_configs.yml
│   └── deploy_loopback.yml
└── inventories/
    ├── netbox.yml
    └── lab/
        ├── hosts.yml
        ├── group_vars/
        │   └── all.yml
        └── host_vars/
            ├── R1-BRANCH.yml
            ├── R2-HQ.yml
            └── R3-DC.yml
```

4.3 Python & Ansible Packages

```
pip install --upgrade pip
pip install ansible "paramiko<4" ncclient pyats
pip install ansible-pylibssh
pip install netaddr
```

Why ansible-pylibssh?

Paramiko (the default SSH library) can have compatibility issues with IOS-XE. ansible-pylibssh provides a more stable SSH transport layer for network devices.

4.4 ansible.cfg

Open the file in nano and paste the config below

```
nano ansible.cfg
```

Paste the following content, then save with Ctrl+O and exit with Ctrl+X:

```
[defaults]
inventory = inventories/lab
host_key_checking = False
retry_files_enabled = False
timeout = 30
interpreter_python = /home/adminuser/automation-venv/bin/python3

[persistent_connection]
connect_timeout = 30
command_timeout = 30
```

4.5 Baseline hosts.yml

Open the file in nano and paste the inventory below

```
nano inventories/lab/hosts.yml
```

Paste the following content, then save with Ctrl+O and exit with Ctrl+X. This is the static inventory used for all lab phases before NetBox dynamic inventory is introduced in Section 6.

```
all:
  children:
    routers:
      hosts:
        R1-BRANCH:
          ansible_host: 10.1.6.101
        R2-HQ:
          ansible_host: 10.1.6.102
        R3-DC:
          ansible_host: 10.1.6.103
```

4.6 Verify Reachability and Ansible Inventory

Before running any playbooks, verify both IP reachability and SSH connectivity from the automation box. Then confirm Ansible can parse the inventory correctly.

Step 1 - Test basic IP reachability

From the automation box, ping each router management IP. If ICMP is blocked on the routers that is acceptable, but SSH must work.

```
ping -c 3 10.1.6.101
ping -c 3 10.1.6.102
ping -c 3 10.1.6.103
```

Step 2 - Test SSH manually

SSH into each router and accept the host key when prompted. If SSH fails, stop here and resolve the connectivity issue before proceeding. Ansible uses SSH for all communication.

```
ssh admin@10.1.6.101
ssh admin@10.1.6.102
ssh admin@10.1.6.103
```

If SSH fails

Check that GigabitEthernet4 is up and has the correct IP on the router. Verify the management cloud in EVE-NG is connected to both the router and the AutomationBox. Confirm the router has the SSH baseline config applied, crypto key, ip ssh version 2, and line vty transport input ssh.

Step 3 - Test Ansible inventory

Run the following commands to confirm Ansible reads the inventory correctly and can resolve each host:

```
ansible-inventory -i inventories/lab/hosts.yml --graph

# Show variables resolved for each host
ansible-inventory -i inventories/lab --host R1-BRANCH
ansible-inventory -i inventories/lab --host R2-HQ
ansible-inventory -i inventories/lab --host R3-DC
```

Expected output from ansible-inventory --graph:

```
@all:
  |--@routers:
  |   |--R1-BRANCH
  |   |--R2-HQ
  |   |--R3-DC
  |--@ungrouped:
```

All three checks must pass

IP reachability (or confirmed ICMP-blocked but SSH works), successful manual SSH to each router, and clean ansible-inventory --graph output showing all three hosts under the routers group.

Step 4 - Run your first ad-hoc commands

With inventory verified and SSH working, ad-hoc commands confirm that Ansible can actually talk to the routers using the IOS modules. This is also a good moment to understand the four inventory source patterns you will use throughout this lab.

#	Inventory Source	What Ansible does
1	Default (ansible.cfg)	Uses the inventory defined in ansible.cfg which is the inventories/lab directory in this project
2	Specific file: -i inventories/lab/hosts.yml	Reads a single YAML file only which is useful when you want to target one file explicitly
3	Directory: -i inventories/lab	Parses all supported files in the folder and merges them which allows splitting inventory across multiple files
4	Dynamic: -i inventories/netbox.yml	Runs the NetBox inventory plugin which queries the NetBox API in real time instead of reading a static file

Run the following ad-hoc commands. Each uses a different inventory source. Notice that the result is identical regardless of which source you use, as long as the routers group resolves correctly.

Using default inventory (from ansible.cfg):

```
ansible routers -m cisco.ios.ios_command -a "commands='show ip interface
brief'"
```

Using a specific inventory file:

```
ansible routers -i inventories/lab/hosts.yml -m cisco.ios.ios_command -a
"commands='show ip interface brief'"
```

Using an inventory directory:

```
ansible routers -i inventories/lab -m cisco.ios.ios_command -a "commands='show
ip interface brief'"
```

Using the dynamic NetBox inventory (after Section 5-6 are complete):

```
ansible routers -i inventories/netbox.yml -m cisco.ios.ios_command -a
"commands='show ip interface brief'"
```

Expected result

Each router returns a table showing all interfaces, their IP addresses, and their status (up/up, up/down, etc.). If any router returns an error instead of output, stop and resolve the connectivity or credential issue before proceeding to Section 5.

Then run a second ad-hoc command to verify device version information:

```
ansible routers -m cisco.ios.ios_command -a "commands='show version'"
```

Expected result: each router returns its IOS-XE version string, hostname, uptime, and hardware model. This confirms the cisco.ios collection is installed correctly and the connection plugin is working end-to-end.

4.7 group_vars/all.yml

Open the file in nano and paste the variables below

```
nano inventories/lab/group_vars/all.yml
```

This file holds shared variables that apply to all hosts. At this stage it only needs the Ansible connection settings and device credentials. Paste the following content, then save with Ctrl+O and exit with Ctrl+X:

```
# Ansible connection — required for cisco.ios modules
ansible_network_os: cisco.ios.ios
ansible_connection: network_cli
ansible_user: "admin"
ansible_password: "Cisco123!"
ansible_ssh_common_args: "-o PreferredAuthentications=password"
```

This file grows as the lab progresses

Additional variables are added to group_vars/all.yml at each phase: Both the OSPF parameters and the NetBox API URL and token are introduced together in Section 7.5 (Lab 5 - Level 5: Ansible + NetBox), after NetBox is deployed and a token is generated. Do not add those values yet, they will be introduced with the context that explains them.

5. NetBox Deployment

NetBox is an open-source infrastructure resource modeling tool. In this lab it serves as the Source of Truth; the authoritative database for all device, interface, and IP address information that drives automation.

5.1 Install NetBox via Docker

```
sudo apt install -y docker.io docker-compose
sudo apt install -y docker-compose-plugin
sudo systemctl enable docker
sudo systemctl start docker
git clone https://github.com/netbox-community/netbox-docker.git
cd netbox-docker
```

Before starting, edit docker-compose.yml to expose the NetBox web UI on port 8000:

```
# In docker-compose.yml under the netbox service:
services:
  netbox:
    ports:
      - "8000:8080"
```

```
sudo docker-compose up -d

# Verify all containers are healthy (may take 15-20 minutes on first start)
sudo docker-compose ps

# create user account and password
sudo docker-compose exec netbox /opt/netbox/netbox/manage.py createsuperuser
```

5.2 Create API Token

Log into the NetBox UI at http://10.1.6.10:8000, navigate to Admin > Users > Tokens, and create a new token. Set the user as a Superuser to avoid object-level permission errors.

Verify the token works from the command line:

```
curl -s http://127.0.0.1:8000/api/dcim/devices/ \
  -H "Authorization: Bearer <your_token>" | python3 -m json.tool | head -20
```

Important: Token Format

NetBox 4.x uses v2 Bearer tokens (format: nbt_xxxx). Older v1 token format will return a 403 'Invalid v1 token' error. Always use 'Bearer <token>' in the Authorization header, not 'Token <token>'.

5.3 Populate NetBox Data Model

The following data must be entered in NetBox before the dynamic inventory and intent-based playbook will work. This can be done through the UI or via the API.

- Create a Site (e.g., 'lab')
- DC
- HQ
- BRANCH
- Create a Manufacturer (e.g., 'cisco')
- Create a Device Role (e.g., 'router')
- Create a Device Type (e.g., 'c8000v')
- Create each Device: R1-BRANCH, R2-HQ, R3-DC - assign site, role, type

Name	Site
R1-BRANCH	BRANCH
R2-HQ	HQ
R3-DC	DC

- Create Interfaces on each device: GigabitEthernet1-4, Loopback0
- **R1-BRANCH**
  - Gi1 → 10.0.12.1/30
  - Gi2 → 10.0.13.1/30
  - Gi3 → 192.168.10.1/24
  - Gi4 → 10.1.6.101/24
  - Loopback0 → 1.1.1.1
- **R2-HQ**
  - Gi1 → 10.0.12.2/30
  - Gi2 → 10.0.23.1/30
  - Gi3 → 192.168.20.1/24
  - Gi4 → 10.1.6.102/24
  - Loopback0 → 2.2.2.2
- **R3-DC**
  - Gi1 → 10.0.23.2/30
  - Gi2 → 10.0.13.2/30
  - Gi3 → 192.168.30.1/24
  - Gi4 → 10.1.6.103/24
  - Loopback0 → 3.3.3.3
- Create IP Addresses and assign them to interfaces
- Create and apply the tag 'ansible-managed' to all interfaces and devices that should be automated
- Set the Primary IPv4 address for each device to be GigabitEthernet4 (Devices -> select device -> edit device)

6. Dynamic NetBox Inventory (Lab 8)

The NetBox inventory plugin replaces the static hosts.yml file. Ansible queries the NetBox API at runtime to build the inventory which means any device added to NetBox is automatically available to Ansible without touching any YAML files.

6.1 Install NetBox Ansible Plugin

Before configuring the inventory file, install the required Python packages for the NetBox inventory plugin:

```
pip install pynetbox
pip install pytz pynetbox
```

6.2 inventories/netbox.yml

Make sure you are in the network-automation directory, then open the file:

```
nano inventories/netbox.yml
```

Paste the following, then save with Ctrl+O and exit with Ctrl+X:

```
plugin: netbox.netbox.nb_inventory
api_endpoint: http://127.0.0.1:8000
token:
  type: Bearer
  value: <your_token_here>
validate_certs: false
group_by:
  - sites
  - device_roles
query_filters:
  - tag: ansible-managed
compose:
  ansible_connection: "'network_cli'"
  ansible_network_os: "'cisco.ios.ios'"
  ansible_user: "'admin'"
  ansible_password: "'Cisco123!'"
  ansible_host: primary_ip.address | ansible.utils.ipaddr('address')
```

Why the compose: block?

The NetBox inventory plugin sets ansible_connection to 'ssh' by default, overriding any value in group_vars. The compose: block forces the correct values at inventory resolution time, before Ansible tries to connect. The double-quoting syntax ("'value'") is required; outer quotes for YAML, inner quotes to produce a literal string.

6.3 Verify Dynamic Inventory

```
# List all hosts from NetBox
ansible all -i inventories/netbox.yml --list-hosts
ansible-inventory -i inventories/netbox.yml --graph

# Verify connection variables are correct
ansible all -i inventories/netbox.yml -m debug -a "var=ansible_connection"
ansible all -i inventories/netbox.yml -m debug -a "var=ansible_user"

# Verify NetBox variables are in scope
ansible all -i inventories/netbox.yml -m debug -a "var=netbox_api_url"
```

Expected inventory groups from NetBox:

- @sites_branch → R1-BRANCH
- @sites_hq → R2-HQ
- @sites_dc → R3-DC

7. The Automation Progression - Theory & Context

Before diving into the lab mechanics, it is worth understanding why the progression from manual CLI to intent-based networking matters. Being able to articulate each level — what it solves, what it still gets wrong, and why the next level exists — is what separates someone who followed a lab guide from someone who genuinely understands automation architecture.

Configuring OSPF at each level will be the use case demonstrated to support the point. Keeping the task constant makes the progression obvious. You can see exactly what changes at each level and why.

How the levels connect to the labs

The levels in this section are the conceptual framework. The labs in this guide are the practical implementation of that framework. Lab 1 operates at Level 1. Lab 2 operates at Level 2. Lab 3 operates at Level 3. Lab 4 operates at Level 4. Labs 5, 8, and 9 operate at Level 5. Labs 10 through 13 pushes deeper into Level 5 and bridge toward Level 6. Section 12 shows the complete phase-to-level mapping.

7.1 Lab 1 - Level 1: Manual CLI

You SSH into each router individually and type every command by hand. Every router. Every time. Below is the complete OSPF configuration required for all three routers in this lab — notice that you must repeat the entire process three times with no reuse.

R1-BRANCH:

```
router ospf 1
 router-id 1.1.1.1
 network 10.0.12.0 0.0.0.3 area 0
 network 10.0.13.0 0.0.0.3 area 0
 network 192.168.10.0 0.0.0.255 area 0
 network 1.1.1.1 0.0.0.0 area 0
 passive-interface GigabitEthernet3
exit
```

R2-HQ:

```
router ospf 1
 router-id 2.2.2.2
 network 10.0.12.0 0.0.0.3 area 0
 network 10.0.23.0 0.0.0.3 area 0
 network 192.168.20.0 0.0.0.255 area 0
 network 2.2.2.2 0.0.0.0 area 0
 passive-interface GigabitEthernet3
exit
```

R3-DC:

```
router ospf 1
 router-id 3.3.3.3
 network 10.0.13.0 0.0.0.3 area 0
 network 10.0.23.0 0.0.0.3 area 0
 network 192.168.30.0 0.0.0.255 area 0
 network 3.3.3.3 0.0.0.0 area 0
 passive-interface GigabitEthernet3
exit
```

That is 24 individual commands across three SSH sessions typed by hand, with no validation, no error handling, and no record of what was done. A single typo silently produces an inconsistent network.

> **Real-world context**
> Most network engineers start here. It is also where a lot of organizations stay — which is precisely why automation skills command a premium in the job market.

Before moving to Level 2 - Remove OSPF config

SSH into each router and remove the OSPF configuration so the next level starts from a clean state:

```
! Run on R1-BRANCH, R2-HQ, and R3-DC
conf t
no router ospf 1
end
write memory

! Verify OSPF is gone
show run | section router ospf
! Should return nothing
```

7.2 Lab 2 - Level 2: Scripts (Ad-hoc Automation)

You write code in any scripting language that SSH's into each router and sends the same commands automatically. This could be Python with Paramiko or Netmiko, Bash with SSH or expect, NAPALM, or older tools like Perl or TCL. The language does not matter. What defines this level is that you are writing raw code to push CLI commands with no automation framework underneath it.

Why Paramiko for this example:

Paramiko is used here deliberately because it is the most transparent illustration of what Level 2 actually involves. Paramiko is a raw SSH library which gives you nothing except a socket connection to the device. You are responsible for opening the shell, timing each command, waiting for the prompt, and closing the session yourself. Every line of the script is plumbing you

wrote by hand. This makes the contrast with Level 3 unmistakable: all of that plumbing disappears when you move to Ansible.

Netmiko is a more practical real-world choice at this level as it understands network device prompts and handles timing automatically, making scripts less fragile. But using Netmiko here would obscure some of the manual work that Level 2 requires, because Netmiko already abstracts some of the same things Ansible handles. Paramiko shows the full cost of this approach more honestly.

Complete Python/Paramiko script — save as deploy_ospf.py:

```
import paramiko
import time

routers = {
    "R1-BRANCH": {
        "ip": "10.1.6.101",
        "commands": [
            "conf t",
            "router ospf 1",
            "router-id 1.1.1.1",
            "network 10.0.12.0 0.0.0.3 area 0",
            "network 10.0.13.0 0.0.0.3 area 0",
            "network 192.168.10.0 0.0.0.255 area 0",
            "network 1.1.1.1 0.0.0.0 area 0",
            "passive-interface GigabitEthernet3",
            "end", "write memory"
        ]
    },
    "R2-HQ": {
        "ip": "10.1.6.102",
        "commands": [
            "conf t",
            "router ospf 1",
            "router-id 2.2.2.2",
            "network 10.0.12.0 0.0.0.3 area 0",
            "network 10.0.23.0 0.0.0.3 area 0",
            "network 192.168.20.0 0.0.0.255 area 0",
            "network 2.2.2.2 0.0.0.0 area 0",
            "passive-interface GigabitEthernet3",
            "end", "write memory"
        ]
    },
    "R3-DC": {
        "ip": "10.1.6.103",
        "commands": [
            "conf t",
            "router ospf 1",
            "router-id 3.3.3.3",
            "network 10.0.13.0 0.0.0.3 area 0",
            "network 10.0.23.0 0.0.0.3 area 0",
            "network 192.168.30.0 0.0.0.255 area 0",
            "network 3.3.3.3 0.0.0.0 area 0",
            "passive-interface GigabitEthernet3",
            "end", "write memory"
        ]
    }
}
```

```
for name, router in routers.items():
    print(f"Configuring {name}...")
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(router["ip"], username="admin", password="Cisco123!")
    shell = ssh.invoke_shell()
    time.sleep(1)
    for cmd in router["commands"]:
        shell.send(cmd + "\n")
        time.sleep(0.5)
    ssh.close()
    print(f"  Done.")
```

To deploy — run from the automation box with the virtual environment active:

```
# Install Paramiko if not already present
pip install paramiko

# Save the script as deploy_ospf.py, then run:
python3 deploy_ospf.py

# Expected output:
# Configuring R1-BRANCH...
#   Done.
# Configuring R2-HQ...
#   Done.
# Configuring R3-DC...
#   Done.
```

The improvement: one command configures all three routers. The router-specific data is embedded in the script as a dictionary.

The problem: every router's OSPF networks are still hardcoded inside the script. There is no error handling; if R2 fails, the script continues silently to R3 without alerting you. There is no idempotency; run it twice and OSPF statements may duplicate. The script has no awareness of current device state. If a new engineer adds a LAN subnet, they must edit the script itself, understand its structure, and hope they do not break the logic for the other routers.

> **Real-world context**
> This is where a lot of self-taught automation lives. It is better than nothing but does not scale. Scripts become unmaintainable quickly and are usually owned by one person who becomes a single point of failure.

Before moving to Level 3 - Remove OSPF config

Create a quick cleanup script to remove OSPF from all three routers, then verify it is gone:

```
# cleanup_ospf.py
import paramiko, time

routers = {
```

```
    "R1-BRANCH": "10.1.6.101",
    "R2-HQ":     "10.1.6.102",
    "R3-DC":     "10.1.6.103"
}

for name, ip in routers.items():
    print(f"Removing OSPF from {name}...")
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(ip, username="admin", password="Cisco123!")
    shell = ssh.invoke_shell()
    time.sleep(1)
    for cmd in ["conf t", "no router ospf 1", "end", "write memory"]:
        shell.send(cmd + "\n")
        time.sleep(0.5)
    ssh.close()
    print(f"  Done.")
```

```
python3 cleanup_ospf.py
```

7.3 Lab 3 - Level 3: Ansible Playbooks (Task-Based Automation)

You replace the ad-hoc script with a structured Ansible playbook using purpose-built IOS modules. This is what separates Level 3 from Level 2; you are no longer writing raw SSH code. The cisco.ios.ios_config module understands IOS syntax natively, checks existing state before applying changes, and produces a structured run report. Below is a complete playbook for all three routers. Note that all router-specific data is still hardcoded and that problem is solved at Level 4.

Save the playbook as deploy_ospf.yml:

```
nano playbooks/deploy_ospf.yml
```

Paste the following content, then save with Ctrl+O and exit with Ctrl+X:

```
---
- name: Configure OSPF on all routers
  hosts: routers
  gather_facts: no

  tasks:
    - name: Configure OSPF on R1-BRANCH
      cisco.ios.ios_config:
        lines:
          - router-id 1.1.1.1
          - network 10.0.12.0 0.0.0.3 area 0
          - network 10.0.13.0 0.0.0.3 area 0
          - network 192.168.10.0 0.0.0.255 area 0
          - network 1.1.1.1 0.0.0.0 area 0
          - passive-interface GigabitEthernet3
        parents: router ospf 1
      when: inventory_hostname == "R1-BRANCH"
```

```
- name: Configure OSPF on R2-HQ
  cisco.ios.ios_config:
    lines:
      - router-id 2.2.2.2
      - network 10.0.12.0 0.0.0.3 area 0
      - network 10.0.23.0 0.0.0.3 area 0
      - network 192.168.20.0 0.0.0.255 area 0
      - network 2.2.2.2 0.0.0.0 area 0
      - passive-interface GigabitEthernet3
    parents: router ospf 1
  when: inventory_hostname == "R2-HQ"

- name: Configure OSPF on R3-DC
  cisco.ios.ios_config:
    lines:
      - router-id 3.3.3.3
      - network 10.0.13.0 0.0.0.3 area 0
      - network 10.0.23.0 0.0.0.3 area 0
      - network 192.168.30.0 0.0.0.255 area 0
      - network 3.3.3.3 0.0.0.0 area 0
      - passive-interface GigabitEthernet3
    parents: router ospf 1
  when: inventory_hostname == "R3-DC"
```

The improvement: Ansible handles SSH connections, retries failed tasks, checks whether config already exists before applying it (idempotency), and produces a structured run report showing changed vs ok vs failed for every task on every host. Run this ten times and the result is always the same.

The problem: the playbook has three near-identical task blocks, one per router, all with hardcoded networks. Adding a fourth router means adding a fourth task block. Changing OSPF process ID means editing every task. The data and the logic are fused. This playbook cannot be handed to someone else to use on a different topology without rewriting it.

Run the playbook

```
ansible-playbook -i inventories/lab playbooks/deploy_ospf.yml
```

Verify OSPF is running on all three routers:

```
! On any router:
show ip ospf neighbor
show ip route ospf
```

> **Real-world context**
> This is professional-grade automation. Most teams that have adopted Ansible operate at this level. It is a significant step up from scripting, but it remains brittle at scale because the data lives inside the automation logic.

Before moving to Level 4 — Remove OSPF config

Use an Ansible ad-hoc command to remove OSPF from all routers in one shot:

```
ansible routers -i inventories/lab -m cisco.ios.ios_config \
  -a "lines='no router ospf 1' match=none"

# Then save the config on all routers
ansible routers -i inventories/lab -m cisco.ios.ios_command \
  -a "commands='write memory'"

# Verify OSPF is removed
ansible routers -i inventories/lab -m cisco.ios.ios_command \
  -a "commands='show run | section router ospf'"
# Each router should return empty output
```

7.4 Lab 4 - Level 4: Ansible + Templates (Abstraction)

You separate the data (which networks, which router-id) from the logic (how to construct the OSPF config). The Jinja2 template holds the logic. The inventory holds the data. The playbook becomes a single generic task that works for every router without modification.

Create the host_vars files for each router:

```
nano inventories/lab/host_vars/R1-BRANCH.yml
```

R1-BRANCH.yml — data only, no logic:

```
ospf:
  process_id: 1
  router_id: 1.1.1.1
  passive_interfaces:
    - GigabitEthernet3
  networks:
    - { network: 10.0.12.0, wildcard: 0.0.0.3, area: 0 }
    - { network: 10.0.13.0, wildcard: 0.0.0.3, area: 0 }
    - { network: 192.168.10.0, wildcard: 0.0.0.255, area: 0 }
    - { network: 1.1.1.1, wildcard: 0.0.0.0, area: 0 }
```

```
nano inventories/lab/host_vars/R2-HQ.yml
```

R2-HQ.yml:

```
ospf:
  process_id: 1
  router_id: 2.2.2.2
  passive_interfaces:
    - GigabitEthernet3
  networks:
    - { network: 10.0.12.0, wildcard: 0.0.0.3, area: 0 }
    - { network: 10.0.23.0, wildcard: 0.0.0.3, area: 0 }
    - { network: 192.168.20.0, wildcard: 0.0.0.255, area: 0 }
    - { network: 2.2.2.2, wildcard: 0.0.0.0, area: 0 }
```

```
nano inventories/lab/host_vars/R3-DC.yml
```

R3-DC.yml:

```
ospf:
  process_id: 1
  router_id: 3.3.3.3
  passive_interfaces:
    - GigabitEthernet3
  networks:
    - { network: 10.0.13.0, wildcard: 0.0.0.3, area: 0 }
    - { network: 10.0.23.0, wildcard: 0.0.0.3, area: 0 }
    - { network: 192.168.30.0, wildcard: 0.0.0.255, area: 0 }
    - { network: 3.3.3.3, wildcard: 0.0.0.0, area: 0 }
```

```
nano templates/ospf.j2
```

templates/ospf.j2 — logic only, no data:

```
router ospf {{ ospf.process_id }}
 router-id {{ ospf.router_id }}
{% for net in ospf.networks %}
 network {{ net.network }} {{ net.wildcard }} area {{ net.area }}
{% endfor %}
{% for iface in ospf.passive_interfaces %}
 passive-interface {{ iface }}
{% endfor %}
```

Save the playbook as deploy_ospf_from_template.yml:

```
nano playbooks/deploy_ospf_from_template.yml
```

Paste the following content, then save with Ctrl+O and exit with Ctrl+X:

```
---
- name: Deploy OSPF via template
  hosts: routers
  gather_facts: no

  tasks:
    - name: Render and apply OSPF configuration
      cisco.ios.ios_config:
        src: ../templates/ospf.j2
```

The improvement: the playbook is now a single generic task identical for every router. Add R4? Create host_vars/R4-SITE.yml with its data and run the same playbook unchanged. The Jinja2 template iterates through whatever networks and passive interfaces are defined in the vars file for each host.

The problem: the data lives in YAML files checked into a Git repo. There is no enforcement; nothing stops someone creating a host_vars file with a duplicate IP or an invalid subnet. There is no single place to query the state of the network. If the YAML files drift out of sync with what is actually on the routers, nobody knows.

Run the playbook

```
ansible-playbook -i inventories/lab playbooks/deploy_ospf_from_template.yml
```

Verify OSPF neighbors are forming and routes are present:

```
! On any router:
show ip ospf neighbor
show ip route ospf
```

> **Real-world context**
> This is where strong automation teams operate. Jinja2 templates are industry standard. If you can do this fluently, you are ahead of most network engineers. This is also the level where automation becomes something you can hand to a colleague without rewriting it.

Before moving to Level 5 - Remove OSPF config

Use the same Ansible ad-hoc command to remove OSPF before switching to the NetBox-driven approach:

```
ansible routers -i inventories/lab -m cisco.ios.ios_config \
  -a "lines='no router ospf 1' match=none"

ansible routers -i inventories/lab -m cisco.ios.ios_command \
  -a "commands='write memory'"

# Confirm all clear
ansible routers -i inventories/lab -m cisco.ios.ios_command \
  -a "commands='show ip ospf neighbor'"
# Should show no neighbors
```

7.5 Lab 5 - Level 5: Ansible + NetBox (Source of Truth)

What "zero hardcoding" means: throughout this document, the phrase refers to eliminating all topology-specific values from playbooks and variable files such as IP addresses, router IDs, OSPF network statements, interface names, NTP server addresses, and configuration parameters. At Level 5, none of these values exist anywhere in the automation code. They are all queried from NetBox at runtime. The playbook expresses only what to do, never what specific values to use. The same playbook runs unchanged on any topology, at any scale, without modification.

Instead of reading data from YAML files, Ansible queries NetBox via API at runtime. NetBox is the database of record for every device, interface, and IP address. There are no host_vars files.

The inventory itself is generated dynamically from NetBox. The playbook queries the API for each router's interfaces and IPs, builds OSPF network statements from live data, and pushes the config with zero hardcoded network addresses anywhere.

Make sure you are in the network-automation directory, then open the file:

```
nano inventories/netbox.yml
```

inventories/netbox.yml — replaces all host_vars YAML files:

```
plugin: netbox.netbox.nb_inventory
api_endpoint: http://127.0.0.1:8000
token:
  type: Bearer
  value: <your_token>
validate_certs: false
group_by:
  - sites
  - device_roles
query_filters:
  - tag: ansible-managed
compose:
  ansible_connection: "'network_cli'"
  ansible_network_os: "'cisco.ios.ios'"
  ansible_user: "'admin'"
  ansible_password: "'Cisco123!'"
```

group_vars/all.yml — only operational parameters, no topology data:

```
netbox_api_url: "http://127.0.0.1:8000"
netbox_token: "<YOUR TOKEN>"
ospf_process_id: 1
ospf_area: 0
ansible_network_os: cisco.ios.ios
ansible_connection: network_cli
ospf_exclude_interfaces:
  - GigabitEthernet4
ansible_user: "admin"
ansible_password: "Cisco123!"
ansible_ssh_common_args: "-o PreferredAuthentications=password"
```

Make sure you are in the network-automation directory, then open the playbook:

```
nano playbooks/deploy_ospf_from_netbox.yml
```

Paste the following content, then save with Ctrl+O and exit with Ctrl+X:

deploy_ospf_from_netbox.yml — complete playbook:

```
---
- name: Build OSPF from NetBox data
  hosts: all
  gather_facts: no
```

```
  vars_files:
    - ../group_vars/all.yml
  vars:
    host_vars: []
    netbox_api_url: "http://127.0.0.1:8000"
    netbox_token: "<YOUR TOKEN>"

  tasks:
    - name: Query device interfaces from NetBox
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/dcim/interfaces/?
device={{ inventory_hostname }}&limit=100"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: netbox_interfaces

    - name: Query IP addresses from NetBox
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/ipam/ip-addresses/?
device={{ inventory_hostname }}&limit=100"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: netbox_ips

    - name: Build interface to IP map
      ansible.builtin.set_fact:
        interface_ip_map: "{{ interface_ip_map | default({}) |
combine({ item.assigned_object.name: item.address }) }}"
      loop: "{{ netbox_ips.json.results }}"
      when:
        - item.assigned_object is defined
        - item.assigned_object is not none
        - item.assigned_object.name is defined

    - name: Build OSPF network lines from NetBox IPs
      ansible.builtin.set_fact:
        ospf_network_lines: "{{ ospf_network_lines | default([]) +
[network_line] }}"
      vars:
        exclude_list: "{{ ospf_exclude_interfaces |
default(['GigabitEthernet4']) }}"
        ospf_area: "{{ ospf_area_id | default('0') }}"
        cidr_ip: "{{ interface_ip_map[item.name] | default('') }}"
        ip_only: "{{ cidr_ip.split('/')[0] if cidr_ip else '' }}"
        net_addr: "{{ cidr_ip | ansible.utils.ipaddr('network') if cidr_ip else
'' }}"
        wildcard: "{{ cidr_ip | ansible.utils.ipaddr('hostmask') if cidr_ip
else '' }}"
        network_line: "network {{ ip_only if item.name.startswith('Loopback')
else net_addr }} {{ '0.0.0.0' if item.name.startswith('Loopback') else wildcard
}} area {{ ospf_area }}"
      loop: "{{ netbox_interfaces.json.results }}"
      when:
```

```
        - item.name not in ospf_exclude_interfaces |
default(['GigabitEthernet4'])
        - item.name in interface_ip_map
        - interface_ip_map[item.name] | default('') != ''

    - name: Derive router ID from Loopback0
      ansible.builtin.set_fact:
        ospf_router_id: "{{ interface_ip_map['Loopback0'].split('/')[0] }}"
      when: "'Loopback0' in interface_ip_map"

    - name: Deploy OSPF configuration to devices
      cisco.ios.ios_config:
        parents: "router ospf 1"
        lines: "{{ ['router-id ' + (ospf_router_id | default(ansible_host))] +
ospf_network_lines }}"
      register: ospf_result
      when: ospf_network_lines | default([]) | length > 0
```

To deploy — run from the automation box with the virtual environment active:

```
ansible-playbook -i inventories/netbox.yml playbooks/deploy_ospf_from_netbox.yml
```

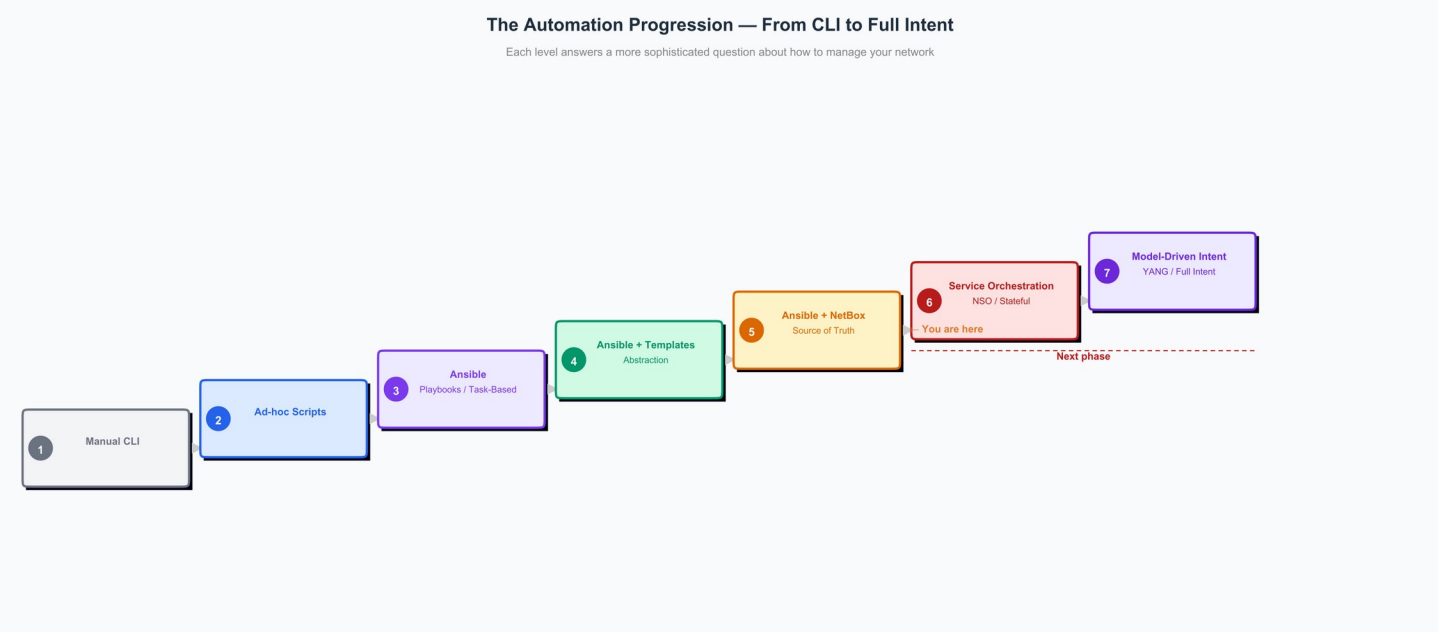
The improvement: there are no YAML files to maintain. The entire topology is described in NetBox. When you add a new device, assign its interfaces and IPs in NetBox, tag it ansible-managed, and run the same playbook it is configured automatically. The playbook itself never changes regardless of how many routers are in the network or how their addressing changes.

What this fundamentally changes: instead of asking what is configured on this router, the question becomes what should be configured according to the data model. The network becomes a reflection of intent, not a collection of manual changes.

Real-world context

This is what you build in Lab 5 of this lab. Organizations at this level have a fundamentally different relationship with their network; automation is no longer a convenience, it is the authoritative mechanism for all changes.

7.6 The Core Mental Model



The progression is really about answering one question at each level. Each level does not replace the previous one — it adds a constraint that forces better thinking:

Level	Stage	The question you are answering
1	Manual CLI	What do I need to type?
2	Scripts (Ad-hoc)	How do I type it faster?
3	Ansible Playbooks	How do I do this consistently across all devices?
4	Ansible + Templates	How do I separate what I want from how to configure it?
5	Ansible + NetBox	How do I make the network reflect reality automatically?
6	Service Orchestration	How do I model services so the system manages state, not just config?
7	Model-Driven Intent	How do I express what I want and let the platform figure out the commands?

Ansible without templates is still brittle. Templates without a source of truth still require manual data maintenance. The full stack working together is what makes automation genuinely resilient and what you built in this lab.

Levels 6 and 7 represent the next frontier. Level 6, Service Orchestration, is where tools like Cisco NSO operate. Rather than writing a playbook that pushes OSPF config and exits, you define an OSPF service with inputs and outputs. NSO maintains the relationship between that service intent and the device config it generated. Delete the service and NSO removes exactly

what it added. A device drifts from intended state and NSO can detect and correct it. Ansible has no memory of what it deployed; NSO does.

Level 7, Model-Driven Intent, sits at the top of the stack. YANG data models define what the network should look like at an abstract level, independent of any vendor's CLI syntax. The orchestrator translates that model into device-specific configuration across any platform automatically. The operator expresses pure intent such as “deploy a BGP peering between these two sites with these policies” and the system determines every command required on every device. This is the direction the industry is heading, and it is the destination that everything in this lab is building toward.

8. Level 3 Labs — Ansible Playbooks (Labs 6 & 7)

These are your first hands-on labs. At this point you have read through the theory in Section 7 and understand that Level 3 is where raw SSH scripting is replaced by structured Ansible playbooks with purpose-built IOS modules. Labs 6 and 7 are classic Level 3 exercises, task-based, using the static inventory from Section 4, with device-specific data still hardcoded in the inventory file.

> **Automation level: Level 3 - Ansible Playbooks**
> These playbooks use the static hosts.yml inventory you built in Section 4. Before running them you need to add the loopback100_ip variable to hosts.yml as noted in Section 4.5. Both playbooks connect directly via network_cli and push config using cisco.ios modules; no templates, no source of truth yet.

8.1 Lab 6 - Loopback Deployment

Configures Loopback100 on each router using host-specific IP variables from the static inventory. This is the first automation test to confirm end-to-end Ansible connectivity with the cisco.ios collection.

Step 1 - Update hosts.yml with loopback100_ip

Before running this playbook, add the loopback100_ip variable to inventories/lab/hosts.yml for each router:

```
all:
  children:
    routers:
      hosts:
        R1-BRANCH:
          ansible_host: 10.1.6.101
          loopback100_ip: 100.100.100.1
        R2-HQ:
          ansible_host: 10.1.6.102
          loopback100_ip: 100.100.100.2
        R3-DC:
          ansible_host: 10.1.6.103
          loopback100_ip: 100.100.100.3
```

Step 2 - Create the playbook

Create and open the playbook file

```
nano playbooks/deploy_loopback.yml
```

Paste the following, then save with Ctrl+O and exit with Ctrl+X:

```
---
- name: Deploy Loopback100 interfaces
  hosts: routers
  gather_facts: no
  tasks:
    - name: Configure Loopback100
      cisco.ios.ios_config:
        parents: interface Loopback100
        lines:
          - ip address {{ loopback100_ip }} 255.255.255.255
          - no shutdown
```

Expected result after running:

Router	Loopback100 IP
R1-BRANCH	100.100.100.1/32
R2-HQ	100.100.100.2/32
R3-DC	100.100.100.3/32

To deploy — run from the automation box with the virtual environment active:

```
ansible-playbook -i inventories/lab playbooks/deploy_loopback.yml
```

8.2 Lab 7 - Configuration Backup

Pulls the running configuration from each router and saves it locally. The directory creation task was added after encountering a 'backups directory does not exist' error which is a common real-world issue.

Create and open the playbook file

```
nano playbooks/backup_configs.yml
```

Paste the following, then save with Ctrl+O and exit with Ctrl+X:

```
---
- name: Backup running configurations
  hosts: routers
  gather_facts: no
  tasks:
    - name: Ensure backups directory exists
      ansible.builtin.file:
        path: "{{ playbook_dir }}/../backups"
        mode: "0755"
        state: directory
      delegate_to: localhost
      run_once: true
```

```
    - name: Retrieve running configuration
      cisco.ios.ios_command:
        commands:
          - show running-config
      register: running_config

    - name: Save running configuration locally
      ansible.builtin.copy:
        content: "{{ running_config.stdout[0] }}"
        dest: "{{ playbook_dir }}/../backups/{{ inventory_hostname }}-running-
config.txt"
      delegate_to: localhost
```

9. Level 5 Labs - NetBox-Driven Configuration (Lab 9)

These labs operate at Level 5 of the automation maturity model; Ansible + NetBox as Source of Truth. Unlike the Level 3 labs in Section 8 where data was hardcoded in the inventory, here the playbooks contain no device-specific data at all. Everything such as NTP server addresses and interface descriptions are pulled live from the NetBox API at runtime.

Automation level: Level 5 — Ansible + NetBox

These playbooks use the dynamic NetBox inventory from Section 6. NetBox is the single source of truth. Changing data in NetBox and rerunning the playbook is all that is required to update the network; no playbook code changes needed.

By this point in the lab, you have built a dynamic inventory from NetBox, connected Ansible to it, and understand how the API-driven approach works. This section applies that same pattern to two practical, production-relevant use cases: NTP configuration and interface descriptions.

OSPF was used extensively in Section 7 to illustrate the automation progression theory. Here we move to different use cases that demonstrate new NetBox concepts, specifically site-level data and interface-level metadata and introduce the cisco.ios.ios_config resource module applied to everyday operational configuration.

What makes this intent-based

Neither playbook in this section contains any NTP server addresses, interface names, or description text. All of that data lives in NetBox. Change it there and the next playbook run reflects the change automatically. The playbook code never needs to be touched.

9.1 NTP Configuration from NetBox

Concept

NTP is one of the most universally required configurations on any network device. Without synchronized time, log correlation, certificate validation, and security auditing all break down. In this lab you will store NTP server addresses in NetBox at the site level, meaning every device at a site inherits the same NTP config, and push it to all three routers automatically.

This introduces a new NetBox concept: site-level data. Rather than querying per-device data as in previous labs, the playbook queries the NetBox site associated with each device and pulls the NTP servers defined there. One change in NetBox updates every router at that site.

Step 1 - Add NTP servers to NetBox

In the NetBox UI navigate to Organization > Sites > your lab site > Edit. Add a custom field named ntp_servers (type: text) and enter the following values separated by commas:

```
# NetBox site custom field: ntp_servers
# Value: 216.239.35.0,216.239.35.4

# Or set via API:
curl -X PATCH http://127.0.0.1:8000/api/dcim/sites/1/ \
  -H "Authorization: Bearer <your_token>" \
  -H "Content-Type: application/json" \
  -d '{"custom_fields": {"ntp_servers": "216.239.35.0,216.239.35.4"}}'
```

Step 2 - Verify the data is in NetBox

```
curl -s http://127.0.0.1:8000/api/dcim/sites/?name=lab \
  -H "Authorization: Bearer <your_token>" | python3 -m json.tool | grep ntp
```

Step 3 - The playbook

Create and open the playbook file

```
nano playbooks/deploy_ntp_from_netbox.yml
```

Paste the following, then save with Ctrl+O and exit with Ctrl+X:

```
---
- name: Deploy NTP configuration from NetBox site data
  hosts: all
  gather_facts: no
  vars_files:
    - ../group_vars/all.yml
  vars:
    host_vars: []
    netbox_api_url: "http://127.0.0.1:8000"
    netbox_token: "<YOUR TOKEN>"
```

```
  tasks:
    - name: Query device from NetBox to get site
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/dcim/devices/?name={{ inventory_hostname
}}"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: netbox_device

    - name: Extract site ID from device result
      ansible.builtin.set_fact:
        device_site_id: "{{ netbox_device.json.results[0].site.id }}"

    - name: Query site data including NTP custom field
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/dcim/sites/{{ device_site_id }}/"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: netbox_site

    - name: Build NTP server list from site custom field
      ansible.builtin.set_fact:
        ntp_servers: "{{ netbox_site.json.custom_fields.ntp_servers.split(',')
| map('trim') | list }}"

    - name: Apply NTP server configuration
      cisco.ios.ios_config:
        lines: "{{ ntp_servers | map('regex_replace', '(.+)', 'ntp server \\1')
| list }}"

    - name: Save configuration
      cisco.ios.ios_command:
        commands: write memory
```

Step 4 - Run the playbook

```
ansible-playbook -i inventories/netbox.yml playbooks/deploy_ntp_from_netbox.yml
```

Step 5 - Validate on the routers

```
show ntp associations
show ntp status

! Expected — associations visible after sync:
  address         ref clock    st  when  poll reach  delay  offset  disp
+~216.239.35.0    .GOOG.        1    23    64   377   1.23   0.451  0.18
+~216.239.35.4    .GOOG.        1    18    64   377   1.31   0.387  0.21
```

What just happened

The playbook queried NetBox for each device, found its site, pulled the NTP servers from a site-level custom field, and pushed ntp server statements to the router. Update the NetBox site field and rerun all routers at that site update instantly. This is site-level intent driving device configuration.

9.2 Lab 9 - Interface Descriptions from NetBox

Concept

Interface descriptions are simple but powerful, they are the first thing an engineer reads during troubleshooting. Consistently populated descriptions that match the NetBox source of truth means documentation is always in sync with reality. This lab pulls the description field from each interface in NetBox and pushes it to the corresponding router interface.

This uses data already in NetBox from earlier lab phases. No new NetBox configuration is required beyond adding description text to interfaces, which you should do in Step 1 before running the playbook.

Step 1 - Set interface descriptions in NetBox

In the NetBox UI navigate to each device and edit its interfaces to add descriptions. Use the following values:

Router	Interface	Description
R1-BRANCH	GigabitEthernet1	WAN-to-R2-HQ-10.0.12.0/30
R1-BRANCH	GigabitEthernet2	WAN-to-R3-DC-10.0.13.0/30
R1-BRANCH	GigabitEthernet3	LAN-BRANCH-192.168.10.0/24
R2-HQ	GigabitEthernet1	WAN-to-R1-BRANCH-10.0.12.0/30
R2-HQ	GigabitEthernet2	WAN-to-R3-DC-10.0.23.0/30
R2-HQ	GigabitEthernet3	LAN-HQ-192.168.20.0/24
R3-DC	GigabitEthernet1	WAN-to-R2-HQ-10.0.23.0/30
R3-DC	GigabitEthernet2	WAN-to-R1-BRANCH-10.0.13.0/30
R3-DC	GigabitEthernet3	LAN-DC-192.168.30.0/24

Step 2 - The playbook

Create and open the playbook file

```
nano playbooks/deploy_interface_descriptions.yml
```

Paste the following, then save with Ctrl+O and exit with Ctrl+X:

```
---
- name: Deploy interface descriptions from NetBox
  hosts: all
  gather_facts: no
  vars_files:
    - ../group_vars/all.yml
  vars:
```

```
    host_vars: []
    netbox_api_url: "http://127.0.0.1:8000"
    netbox_token: "<YOUR TOKEN>"

  tasks:
    - name: Query interfaces from NetBox
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/dcim/interfaces/?
device={{ inventory_hostname }}&limit=100"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: netbox_interfaces

    - name: Apply interface descriptions from NetBox
      cisco.ios.ios_config:
        lines:
          - description {{ item.description }}
        parents: interface {{ item.name }}
      loop: "{{ netbox_interfaces.json.results }}"
      when:
        - item.description is defined
        - item.description | length > 0
        - item.name != "GigabitEthernet4"

    - name: Save configuration
      cisco.ios.ios_command:
        commands: write memory
```

Step 3 - Run the playbook

```
ansible-playbook -i inventories/netbox.yml
playbooks/deploy_interface_descriptions.yml
```

Step 4 - Validate

```
! On any router:
show interfaces description

! Expected output on R1-BRANCH:
Interface         Status   Protocol Description
Gi1               up       up       WAN-to-R2-HQ-10.0.12.0/30
Gi2               up       up       WAN-to-R3-DC-10.0.13.0/30
Gi3               up       up       LAN-BRANCH-192.168.10.0/24
Gi4               up       up
Lo0               up       up
```

Try it yourself

Update one interface description in NetBox and rerun the playbook. The router immediately reflects the change. This is the core value of a source of truth, the device config is no longer the reference, NetBox is.

10. Lab Summary

Lab	What Was Built	Outcome	Automation Level
Lab 1	Manual OSPF configuration	Manual connectivity confirmed. OSPF convergence reached	Level 1 - Manual CLI
Lab 2	Ad-hoc Python Scripts	OSPF convergence reached	Level 2 - Ad-hoc Automation
Lab 3	Configuration OSPF automation	OSPF convergence reached	Level 3 - Task-Based Automation (Ansible Playbooks)
Lab 4	Jinja2 OSPF template + host_vars	Template rendering validated. OSPF convergence reached	Level 4 - Ansible + Templates
Lab 5	Intent-based OSPF from NetBox	OSPF live, neighbors FULL state	Level 5 - Ansible + NetBox
Lab 6	Ansible loopback deployment playbook	Loopback100 on all 3 routers	Level 3 - Task-Based Automation (Ansible Playbooks)
Lab 7	Configuration backup automation	Running configs saved locally	Level 3 - Task-Based Automation (Ansible Playbooks)
Lab 8	NetBox dynamic inventory plugin	Dynamic groups working	Level 5 - Ansible + NetBox
Lab 9	Interface descriptions from NetBox	Description field from each interface in NetBox is pulled	Level 5 - Ansible + NetBox
Lab 10	Full Intent Model — OSPF — zero hardcoding	Playbook driven entirely by NetBox	Level 5 advanced
Lab 11	Compliance checks + tag-based enforcement	Drift reported, roles enforced	Level 5 advanced
Lab 12	CI/CD Pipeline with Git	Automated deploy + validation pipeline	Level 5 / bridge to Level 6
Lab 13	Drift Detection + remediation	Network state matches NetBox intent	Level 5 / bridge to Level 6

Lab 2 - Level 2 in context

Lab 2 is the ad-hoc scripting lab at Level 2. It is a hands-on exercise, not just theory. Section 7.2 walks through a complete Paramiko script that configures OSPF on all three routers. This is the only lab in the guide that does not use Ansible, which is deliberate as it shows exactly what Level 2 costs in terms of fragility, manual effort, and lack of idempotency, making the jump to Level 3 (Lab 3) unmistakable. Labs 10 through 13 rounds out Level 5 maturity and bridge toward Level 6; covered in Lab Guide Part 2.

11. Continuing the Journey - Labs 10 Through 13

Lab 9: Complete. The platform is ready to evolve.

You have built a production-grade network automation platform from scratch. EVE-NG is running a three-router topology. Ansible is connected. NetBox is deployed and serving as the source of truth. Intent-based playbooks are pulling live data from the API and configuring the network automatically. That is Level 5 and it is where most enterprise automation teams aspire to be.

What you have built in this lab is not a demo. It is a real platform. The same architecture, Ansible, NetBox, dynamic inventory, and intent-based playbooks, runs in production environments at organizations managing thousands of devices. The difference between what you have here and what they have is scale, not structure.

Labs 10 through 13 continue in this guide. They push deeper into Level 5 maturity, introduce operational discipline, and build the conceptual bridge toward Level 6. When Lab 13 is complete you will understand exactly where Ansible + NetBox reaches its ceiling and why tools like Cisco NSO exist to go beyond it. Level 6 and Level 7 are covered in Lab Guide Part 2.

Lab 10 - Full Intent Model (Level 5 Advanced)

The OSPF playbook from Lab 5 still has two hardcoded values: the ospf_exclude_interfaces list in group_vars and the ospf_process_id variable. Lab 10 removes both entirely by using NetBox data to drive those decisions.

Step 1 - Mark GigabitEthernet4 as management-only in NetBox

In the NetBox UI navigate to each device, open GigabitEthernet4, and check the Management Only checkbox. Do this for R1-BRANCH, R2-HQ, and R3-DC. This tells the playbook to exclude this interface without any hardcoded list.

Verify via API:

```
curl -s 'http://10.1.6.10:8000/api/dcim/interfaces/?device=R1-
BRANCH&name=GigabitEthernet4' \
  -H 'Authorization: Bearer <your_token>' | python3 -m json.tool | grep
mgmt_only
# Should return: "mgmt_only": true
```

Step 2 - Add ospf_process_id custom field to NetBox devices

In the NetBox UI navigate to Customization > Custom Fields. Create a new custom field with these settings:

Field	Value
Name	ospf_process_id
Label	OSPF Process ID
Type	Integer
Object Type	DCIM > Device
Default	1

Then edit each device (R1-BRANCH, R2-HQ, R3-DC) and set ospf_process_id to 1.

Step 3 - Update group_vars/all.yml

Remove the ospf_process_id, ospf_area, and ospf_exclude_interfaces variables — these now come from NetBox. The file should only contain connection and credential settings:

```
nano inventories/lab/group_vars/all.yml
```

```
netbox_api_url: "http://127.0.0.1:8000"
netbox_token: "<YOUR TOKEN>"
ansible_network_os: cisco.ios.ios
ansible_connection: network_cli
ansible_user: "admin"
ansible_password: "Cisco123!"
ansible_ssh_common_args: "-o PreferredAuthentications=password"
```

Step 4 - Create the updated playbook

```
nano playbooks/deploy_ospf_full_intent.yml
```

```
---
- name: Deploy OSPF — full intent, zero hardcoding
  hosts: all
  gather_facts: no
  vars:
    netbox_api_url: "http://127.0.0.1:8000"
    netbox_token: "<YOUR TOKEN>"

  tasks:
    - name: Query device data from NetBox
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/dcim/devices/?name={{ inventory_hostname
}}"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: netbox_device
```

```
    - name: Extract OSPF process ID from device custom field
      ansible.builtin.set_fact:
        ospf_pid:
"{{ netbox_device.json.results[0].custom_fields.ospf_process_id |
default(1) }}"

    - name: Query interfaces — exclude mgmt_only
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/dcim/interfaces/?
device={{ inventory_hostname }}&mgmt_only=false&limit=100"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: netbox_interfaces

    - name: Query IP addresses from NetBox
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/ipam/ip-addresses/?
device={{ inventory_hostname }}&limit=100"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: netbox_ips

    - name: Build interface to IP map
      ansible.builtin.set_fact:
        interface_ip_map: "{{ interface_ip_map | default({}) |
combine({ item.assigned_object.name: item.address }) }}"
      loop: "{{ netbox_ips.json.results }}"
      when:
        - item.assigned_object is defined
        - item.assigned_object is not none
        - item.assigned_object.name is defined

    - name: Build OSPF network lines
      ansible.builtin.set_fact:
        ospf_network_lines: "{{ ospf_network_lines | default([]) +
[network_line] }}"
      vars:
        cidr_ip: "{{ interface_ip_map[item.name] | default('') }}"
        ip_only: "{{ cidr_ip.split('/')[0] if cidr_ip else '' }}"
        net_addr: "{{ cidr_ip | ansible.utils.ipaddr('network') if cidr_ip else
'' }}"
        wildcard: "{{ cidr_ip | ansible.utils.ipaddr('hostmask') if cidr_ip
else '' }}"
        network_line: "network {{ ip_only if item.name.startswith('Loopback')
else net_addr }} {{ '0.0.0.0' if item.name.startswith('Loopback') else wildcard
}} area 0"
      loop: "{{ netbox_interfaces.json.results }}"
      when:
        - item.name in interface_ip_map
        - interface_ip_map[item.name] | default('') != ''

    - name: Derive router ID from Loopback0
      ansible.builtin.set_fact:
```

```
        ospf_router_id: "{{ interface_ip_map['Loopback0'].split('/')[0] }}"
      when: "'Loopback0' in interface_ip_map"

    - name: Deploy OSPF configuration
      cisco.ios.ios_config:
        parents: "router ospf {{ ospf_pid }}"
        lines: "{{ ['router-id ' + ospf_router_id] + ospf_network_lines }}"
      when: ospf_network_lines | default([]) | length > 0
```

Step 5 - Run and verify

```
ansible-playbook -i inventories/netbox.yml
playbooks/deploy_ospf_full_intent.yml

! Verify on any router:
show ip ospf neighbor
show ip ospf
! OSPF process ID should match NetBox custom field value
```

What this demonstrates

The playbook now contains zero topology data. It expresses only behavior; query NetBox, build OSPF config from what NetBox says, apply it. Move it to a different topology with no code changes. This is the completion of Level 5.

Lab 11 - Compliance Checks and Tag-Based Enforcement (Level 5 Advanced)

Lab 11 adds operational verification to the platform. You build a compliance check playbook that compares intended state (NetBox) against running state (device) and reports any drift. You also enforce that only devices tagged ospf-enabled in NetBox receive OSPF configuration.

Step 1 - Create the ospf-enabled tag in NetBox

In the NetBox UI navigate to Customization > Tags and create a tag named ospf-enabled. Then edit each of R1-BRANCH, R2-HQ, and R3-DC and apply the tag under the Tags field.

Verify via API:

```
curl -s 'http://10.1.6.10:8000/api/dcim/devices/?tag=ospf-enabled' \
  -H 'Authorization: Bearer <your_token>' | python3 -m json.tool | grep name
# Should return all three routers
```

Step 2 - Update netbox.yml to filter by tag

Update inventories/netbox.yml to create a separate group for ospf-enabled devices:

```
nano inventories/netbox.yml
```

```
plugin: netbox.netbox.nb_inventory
api_endpoint: http://127.0.0.1:8000
token:
  type: Bearer
  value: <your_token_here>
validate_certs: false
group_by:
  - sites
  - device_roles
  - tags
query_filters:
  - tag: ansible-managed
compose:
  ansible_connection: "'network_cli'"
  ansible_network_os: "'cisco.ios.ios'"
  ansible_user: "'admin'"
  ansible_password: "'Cisco123!'"
  ansible_host: primary_ip.address | ansible.utils.ipaddr('address')
```

The group_by: tags line creates automatic inventory groups from NetBox tags. Devices tagged ospf-enabled will appear under the group tags_ospf_enabled in the inventory.

```
# Verify the tag group appears
ansible-inventory -i inventories/netbox.yml --graph | grep ospf
# Expected: @tags_ospf_enabled:
#   |--R1-BRANCH
#   |--R2-HQ
#   |--R3-DC
```

Step 3 - Update the OSPF playbook to use the tag group

Edit deploy_ospf_full_intent.yml and change the hosts line so OSPF is only pushed to tagged devices:

```
# Change this line in deploy_ospf_full_intent.yml:
  hosts: all

# To this:
  hosts: tags_ospf_enabled
```

Now only devices with the ospf-enabled tag in NetBox receive OSPF configuration. Remove the tag from a device in NetBox and rerun — that router will be skipped automatically.

Step 4 - Create the compliance check playbook

```
nano playbooks/check_ospf_compliance.yml
```

```
---
- name: OSPF Compliance Check — NetBox vs Running Config
  hosts: tags_ospf_enabled
  gather_facts: no
  vars:
    netbox_api_url: "http://127.0.0.1:8000"
    netbox_token: "<YOUR TOKEN>"
    compliance_results: []

  tasks:
    - name: Query intended router-id from NetBox (Loopback0 IP)
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/ipam/ip-addresses/?
device={{ inventory_hostname }}&interface=Loopback0"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: nb_loopback

    - name: Extract intended router-id
      ansible.builtin.set_fact:
        intended_router_id: "{{ nb_loopback.json.results[0].address.split('/')
[0] }}"
      when: nb_loopback.json.results | length > 0

    - name: Get running OSPF config from device
      cisco.ios.ios_command:
        commands:
          - show run | section router ospf
      register: running_ospf

    - name: Check router-id compliance
      ansible.builtin.set_fact:
        router_id_compliant: "{{ intended_router_id in
running_ospf.stdout[0] }}"
```

```
    - name: Get OSPF neighbor state
      cisco.ios.ios_command:
        commands:
          - show ip ospf neighbor
      register: ospf_neighbors

    - name: Report compliance status
      ansible.builtin.debug:
        msg: >
          {{ inventory_hostname }}:
          router-id={{ 'COMPLIANT' if router_id_compliant else 'DRIFT — 
expected ' + intended_router_id }},
          neighbors={{ ospf_neighbors.stdout[0] | regex_findall('FULL') | 
length }} FULL adjacencies
```

Step 5 - Run the compliance check

```
ansible-playbook -i inventories/netbox.yml playbooks/check_ospf_compliance.yml

# Expected output when all routers are compliant:
# R1-BRANCH: router-id=COMPLIANT, neighbors=2 FULL adjacencies
# R2-HQ: router-id=COMPLIANT, neighbors=2 FULL adjacencies
# R3-DC: router-id=COMPLIANT, neighbors=2 FULL adjacencies

# To test drift detection: SSH into R1-BRANCH and change the router-id
# conf t
# router ospf 1
# router-id 9.9.9.9
# end
# Then rerun the compliance check — R1-BRANCH should show DRIFT
```

What this demonstrates

Compliance checking closes the feedback loop. You now have a platform that can deploy config AND verify that deployed config matches intent. This is the operational discipline that separates a mature automation practice from one that just runs playbooks and hopes.

Lab 12 - CI/CD Pipeline with Git (Level 5 / Bridge to Level 6)

Lab 12 puts the automation platform under version control and enforces a safe deployment sequence. Every change is committed to Git before it touches the network. The pipeline validates, dry-runs, deploys, and verifies in sequence and if any stage fails, the pipeline stops.

Step 1 - Initialize a Git repository

```
cd ~/network-automation
git init
git config user.name "Network Automation"
git config user.email "automation@lab.local"

# Create a .gitignore to exclude sensitive files
nano .gitignore
```

```
# .gitignore
backups/
*.retry
automation-venv/
__pycache__/
```

```
# Commit the current state of the project
git add .
git commit -m 'Initial commit — Phase 12 baseline'
git log --oneline
```

Step 2 - Create the pipeline script

```
nano pipeline.sh
```

```
#!/bin/bash
# pipeline.sh — safe deployment pipeline
# Run from ~/network-automation with venv active

set -e  # Exit immediately on any error
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')
LOG_FILE="backups/pipeline_$(date '+%Y%m%d_%H%M%S').log"

log() { echo "[$TIMESTAMP] $1" | tee -a $LOG_FILE; }

log '=== Pipeline started ===='
log "Git commit: $(git log --oneline -1)"

log 'Stage 1: Validate NetBox API connectivity'
curl -sf http://10.1.6.10:8000/api/ -H 'Authorization: Bearer '"$NETBOX_TOKEN"
> /dev/null \
  && log 'NetBox: OK' || { log 'NetBox: FAILED'; exit 1; }

log 'Stage 2: Validate Ansible inventory from NetBox'
ansible-inventory -i inventories/netbox.yml --list > /dev/null \
  && log 'Inventory: OK' || { log 'Inventory: FAILED'; exit 1; }

log 'Stage 3: Dry run — check mode, no changes applied'
```

```
ansible-playbook -i inventories/netbox.yml
playbooks/deploy_ospf_full_intent.yml --check \
  && log 'Dry run: PASSED' || { log 'Dry run: FAILED — aborting deployment';
exit 1; }

log 'Stage 4: Deploy'
ansible-playbook -i inventories/netbox.yml
playbooks/deploy_ospf_full_intent.yml \
  && log 'Deploy: SUCCESS' || { log 'Deploy: FAILED'; exit 1; }

log 'Stage 5: Post-deployment compliance check'
ansible-playbook -i inventories/netbox.yml
playbooks/check_ospf_compliance.yml \
  && log 'Compliance: PASSED' || { log 'Compliance: FAILED — review output
above'; exit 1; }

log '=== Pipeline complete — all stages passed ==='
```

```
chmod +x pipeline.sh
```

Step 3 - Set the NetBox token as an environment variable

Rather than hardcoding the token in the script, export it as an environment variable before running:

```
export NETBOX_TOKEN='<YOUR TOKEN>'

# Add to your shell profile to persist across sessions:
echo "export NETBOX_TOKEN='<YOUR TOKEN>'" >> ~/.bashrc
```

Step 4 - Run the pipeline

```
cd ~/network-automation
source automation-venv/bin/activate
./pipeline.sh

# Expected output:
# [timestamp] === Pipeline started ===
# [timestamp] Git commit: abc1234 Initial commit — Phase 12 baseline
# [timestamp] Stage 1: Validate NetBox API connectivity
# [timestamp] NetBox: OK
# [timestamp] Stage 2: Validate Ansible inventory from NetBox
# [timestamp] Inventory: OK
# [timestamp] Stage 3: Dry run — check mode, no changes applied
# [timestamp] Dry run: PASSED
# [timestamp] Stage 4: Deploy
# [timestamp] Deploy: SUCCESS
# [timestamp] Stage 5: Post-deployment compliance check
# [timestamp] Compliance: PASSED
# [timestamp] === Pipeline complete — all stages passed ===
```

Step 5 - Commit a change and re-run

Make a change to a playbook, commit it to Git, then run the pipeline to see the full change-management workflow in action:

```
# Make a change, commit it, then deploy via pipeline
git add playbooks/deploy_ospf_full_intent.yml
git commit -m 'Update OSPF playbook — add passive-interface logic'
./pipeline.sh

# Check the pipeline log
ls backups/pipeline_*.log
cat backups/pipeline_<latest>.log
```

What this demonstrates

Every deployment now has an audit trail in Git, a dry-run gate that must pass before anything touches the network, and a compliance verification that runs after deployment. This is change management as code and it's the same discipline used in production CI/CD systems for network automation.

Lab 13 - Drift Detection and Remediation (Level 5 / Bridge to Level 6)

Lab 13 is the final lab in this guide. You build a drift detection playbook that continuously compares the running state of each router against what NetBox says it should be. When drift is detected the playbook either reports it or remediates it automatically, depending on a flag you pass at runtime.

Step 1 - Create the drift detection playbook

```
nano playbooks/detect_drift.yml
```

```
---
- name: OSPF Drift Detection and Remediation
  hosts: tags_ospf_enabled
  gather_facts: no
  vars:
    netbox_api_url: "http://127.0.0.1:8000"
    netbox_token: "<YOUR TOKEN>"
    drift_action: report   # override with -e drift_action=remediate

  tasks:
    - name: Query intended interfaces from NetBox (exclude mgmt_only)
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/dcim/interfaces/?
device={{ inventory_hostname }}&mgmt_only=false&limit=100"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: netbox_interfaces

    - name: Query IP addresses from NetBox
      ansible.builtin.uri:
        url: "{{ netbox_api_url }}/api/ipam/ip-addresses/?
device={{ inventory_hostname }}&limit=100"
        method: GET
        headers:
          Authorization: "Bearer {{ netbox_token }}"
          Accept: "application/json"
        return_content: true
      delegate_to: localhost
      register: netbox_ips

    - name: Build interface to IP map
      ansible.builtin.set_fact:
        interface_ip_map: "{{ interface_ip_map | default({}) |
combine({ item.assigned_object.name: item.address }) }}"
      loop: "{{ netbox_ips.json.results }}"
      when:
        - item.assigned_object is defined
        - item.assigned_object is not none
        - item.assigned_object.name is defined

    - name: Build intended OSPF network lines from NetBox
      ansible.builtin.set_fact:
```

```
        intended_ospf_lines: "{{ intended_ospf_lines | default([]) +
[network_line] }}"
      vars:
        cidr_ip: "{{ interface_ip_map[item.name] | default('') }}"
        ip_only: "{{ cidr_ip.split('/')[0] if cidr_ip else '' }}"
        net_addr: "{{ cidr_ip | ansible.utils.ipaddr('network') if cidr_ip else
'' }}"
        wildcard: "{{ cidr_ip | ansible.utils.ipaddr('hostmask') if cidr_ip
else '' }}"
        network_line: "network {{ ip_only if item.name.startswith('Loopback')
else net_addr }} {{ '0.0.0.0' if item.name.startswith('Loopback') else wildcard
}} area 0"
      loop: "{{ netbox_interfaces.json.results }}"
      when:
        - item.name in interface_ip_map
        - interface_ip_map[item.name] | default('') != ''

    - name: Pull running OSPF config from device
      cisco.ios.ios_command:
        commands:
          - show run | section router ospf
      register: running_ospf

    - name: Detect drift — compare each intended line against running config
      ansible.builtin.set_fact:
        missing_lines: "{{ intended_ospf_lines | default([]) | select('string')
| reject('in', running_ospf.stdout[0]) | list }}"

    - name: Report — log any missing lines
      ansible.builtin.debug:
        msg: "{{ inventory_hostname }}: {{ 'COMPLIANT' if missing_lines |
length == 0 else 'DRIFT DETECTED — missing: ' + missing_lines | join(', ') }}"

    - name: Remediate — push missing lines back to device
      cisco.ios.ios_config:
        parents: "router ospf 1"
        lines: "{{ missing_lines }}"
      when:
        - drift_action == 'remediate'
        - missing_lines | length > 0

    - name: Save config after remediation
      cisco.ios.ios_command:
        commands: write memory
      when:
        - drift_action == 'remediate'
        - missing_lines | length > 0
```

Step 2 - Run in report mode

```
ansible-playbook -i inventories/netbox.yml playbooks/detect_drift.yml -e
drift_action=report

# Expected when all routers are compliant:
# R1-BRANCH: COMPLIANT
# R2-HQ: COMPLIANT
# R3-DC: COMPLIANT
```

Step 3 - Introduce drift manually and detect it

SSH into R1-BRANCH and remove one OSPF network statement to simulate drift:

```
! On R1-BRANCH:
conf t
router ospf 1
no network 192.168.10.0 0.0.0.255 area 0
end
! Do NOT write memory — this simulates an unintended manual change

! Back on the automation box — run drift detection:
ansible-playbook -i inventories/netbox.yml playbooks/detect_drift.yml -e
drift_action=report

# Expected output:
# R1-BRANCH: DRIFT DETECTED — missing: network 192.168.10.0 0.0.0.255 area 0
# R2-HQ: COMPLIANT
# R3-DC: COMPLIANT
```

Step 4 - Remediate automatically

```
ansible-playbook -i inventories/netbox.yml playbooks/detect_drift.yml -e
drift_action=remediate

# Expected output:
# R1-BRANCH: DRIFT DETECTED — remediating...
# changed: [R1-BRANCH]

! Verify on R1-BRANCH:
show run | section router ospf
! network 192.168.10.0 0.0.0.255 area 0 should be back

! Run report mode again to confirm clean state:
ansible-playbook -i inventories/netbox.yml playbooks/detect_drift.yml -e
drift_action=report
# All three routers should now show COMPLIANT
```

Step 5 - Add drift detection to the pipeline

Update pipeline.sh to run drift detection as a final stage after every deployment:

```
# Add this block to the end of pipeline.sh before the final log line:

log 'Stage 6: Drift detection — verify no unintended state'
ansible-playbook -i inventories/netbox.yml playbooks/detect_drift.yml -e
drift_action=report \
  && log 'Drift check: CLEAN' || { log 'Drift check: DRIFT DETECTED — review
above'; exit 1; }
```

What this demonstrates and where it points

You have now built, manually in Ansible, what NSO does natively: a system that knows intended state, detects when reality diverges from it, and can restore it automatically. The difference is that

Ansible must be told to check; NSO watches continuously. That is the architectural leap Lab Guide Part 2 introduces. After Lab 13 you have reached the practical ceiling of the Ansible + NetBox stack and are fully prepared for Level 6.

Where You Stand

Completing all 13 phases places you at the top of Level 5 maturity and on the threshold of Level 6. You have built a platform, deployed config from a source of truth, enforced it with compliance checks, protected it with a CI/CD pipeline, and taught it to detect and heal drift automatically. That is the full stack. Lab Guide Part 2 picks up exactly here with NSO, YANG models, and model-driven intent.

12. Troubleshooting Reference

This section documents the real errors encountered during lab build and their resolutions. These represent genuine enterprise-level debugging patterns.

12.1 Common Errors & Fixes

Error	Root Cause	Fix
'netbox_api_url' is undefined	NetBox inventory plugin overrides group_vars scope	Add vars to playbook vars_files: block and inventories/netbox.yml compose:
Invalid v1 token (403)	Old token or wrong format	Generate new token in NetBox UI; use Bearer format
Connection type ssh not valid	ansible_connection defaults to ssh; ios_config needs network_cli	Add ansible_connection: network_cli to compose: block in netbox.yml
Unable to determine host network os	ansible_network_os not set	Add ansible_network_os: cisco.ios.ios to compose: block
Authentication failed (publickey)	network_cli tries public key before password	Set ansible_user and ansible_password in compose: block
'ospf_exclude_interfaces' is undefined	when: conditions evaluated before task vars: block	Define in playbook-level vars: as default fallback expression
YAML syntax error at vars:	Indentation broken during editing	Every key under the play must be exactly 2 spaces indented
backups directory does not exist	Playbook assumes directory exists	Add file: module task with state: directory before copy task

12.2 Key Architecture Lessons

- Inventory source determines variable scope - variables set by a dynamic inventory plugin override group_vars for that same inventory run
- delegate_to: localhost switches the execution context - variables available on a router host may not be visible in the localhost context
- when: conditions are evaluated before task vars: - never reference task-local variables in when: conditions; use play-level vars or evaluate inline
- The NetBox inventory plugin sets ansible_connection: ssh by default - always override it explicitly in the compose: block
- Always run playbooks from the project root directory - relative paths in templates, vars_files, and inventory all resolve from there

Appendix A - Installing EVE-NG Node Images with FileZilla

This appendix provides step-by-step instructions for connecting to your EVE-NG server and uploading node images using FileZilla, which is the recommended method for transferring image files. The EVE-NG web UI image upload can be unreliable for large files. FileZilla over SFTP is faster and more reliable.

A.1 Prerequisites

- FileZilla installed on your workstation - download free from filezilla-project.org
- EVE-NG server IP address and root credentials
- Node image files downloaded to your workstation

A.2 Connect to EVE-NG via FileZilla

Open FileZilla and connect to the EVE-NG server using the following settings:

Field	Value
Protocol	SFTP — SSH File Transfer Protocol
Host	Your EVE-NG server IP address
Port	22
Logon Type	Normal
User	root
Password	Your EVE-NG root password

Click Quickconnect. Accept the server host key if prompted. You should see the EVE-NG server filesystem in the right panel.

A.3 Upload C8000V Router Image

The C8000V image must be placed in the correct directory on the EVE-NG server. Navigate to the following path in the FileZilla remote panel:

```
/opt/unetlab/addons/qemu/
```

Create a new folder named exactly: 8000V-17.13.01-CML

```
# Folder name must match exactly:
8000V-17.13.01-CML
```

Upload the C8000V disk image file (typically named something like c8000v-universalk9_2LPE.17.13.01a.qcow2) into this folder. When the upload is complete, SSH into the EVE-NG server and run the fix permissions command:

```
ssh root@<your-eve-ng-ip>
/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

A.4 Upload Catalyst 9KV Switch Image

Navigate to the same base directory in FileZilla:

```
/opt/unetlab/addons/qemu/
```

Create a new folder named exactly: CAT9KV-17.13.1

Upload the Catalyst 9KV disk image into this folder, then run fix permissions again:

```
/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

A.5 Upload Ubuntu Linux Image (AutomationBox)

The Ubuntu 22.04 server image for the AutomationBox uses the Linux template in EVE-NG. Navigate to:

```
/opt/unetlab/addons/qemu/
```

Create a new folder named exactly: linux-ubuntu-22.04-server

Upload the Ubuntu 22.04 disk image (typically named hda.qcow2) into this folder. Run fix permissions:

```
/opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

A.6 Verify Images in EVE-NG

After uploading all images and running fix permissions, verify they appear in the EVE-NG node list:

- In the EVE-NG web UI, open your lab and click Add Node
- Search for C8000V - the 17.13.01 image should appear
- Search for Catalyst 9KV - the 17.13.1 image should appear
- Search for Linux - the ubuntu-22.04-server image should appear
- VPCS requires no image - it appears automatically

Fix permissions must be run after every upload

EVE-NG will not see newly uploaded images until /opt/unetlab/wrappers/unl_wrapper -a fixpermissions has been executed on the server. If an image does not appear in the node list after upload, this command is almost always the fix.